



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Proyecto: Sistema Verificador de Cumplimiento
OWASP**

Curso: Calidad y Pruebas de Software

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

- Andia Navarro, Diego Fabrizio (2022073906)
- Concha Llaca, Gerardo Alejandro (2017057849)

Tacna – Perú

2026

CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	Equipo	Profesor	-	24/06/2026	Versión inicial de los estándares de programación
1.1	Equipo	Profesor	-	02/07/2026	Ampliación de los estándares para cubrir CLI, manejo de errores, esquemas de datos, logs y seguridad OWASP del sistema completo

ÍNDICE

1. Introducción	3
2. Estándares para el Backend (Python)	3
2.1 Formato y Estilo de Código (PEP 8)	3
2.2 Tipado Estático y Modelado de Datos	3
2.3 Arquitectura y Organización del Código	4
3. Estándares para la Herramienta de Línea de Comandos (CLI)	4
4. Estándares para la Extensión de VS Code (JavaScript)	5
4.1 Estilo de JavaScript	5
4.2 Arquitectura de la Extensión	5
5. Estándares para el Frontend (HTML y CSS)	5
5.1 HTML	5
5.2 CSS	6
6. Manejo de Errores y Registro (Logging)	6
7. Control de Seguridad en la Programación (OWASP Aplicado)	7
8. Pruebas y Aseguramiento de Calidad	7
8.1 Pruebas Unitarias	7
8.2 Estándar de Mensajes de Commit (Git)	8

1. Introducción

Este documento establece las directrices de codificación, estilos y buenas prácticas de desarrollo para el Sistema Verificador de Cumplimiento OWASP. El cumplimiento de estas normas garantiza la legibilidad del código, facilita el mantenimiento colaborativo y previene la introducción de vulnerabilidades en el propio sistema.

El proyecto está compuesto por los siguientes componentes clave:

1. Backend (Python + FastAPI)
2. Herramienta de Línea de Comandos (CLI en Python)
3. Extensión de VS Code (JavaScript + HTML/CSS)
4. Base de Datos (MySQL / SQLite3)

2. Estándares para el Backend (Python)

2.1 Formato y Estilo de Código (PEP 8)

Se sigue el estándar oficial PEP 8 con las siguientes precisiones:

- Indentación: Se deben utilizar estrictamente 4 espacios por nivel de anidamiento. No se permiten tabulaciones (\t).
- Límite de caracteres por línea: Un máximo de 100 caracteres por línea para favorecer la lectura vertical en pantallas estándar.
- Nombres de archivos y directorios: Deben escribirse en minúsculas y separados por guiones bajos si es necesario (ej: analysis_service.py).
- Nombres de variables y funciones: En formato snake_case (ej: calculate_score(), scan_store).
- Nombres de clases: En formato PascalCase (ej: InMemoryScanStore, APIToken).
- Nombres de constantes: En mayúsculas y en formato SNAKE_CASE (ej: KNOWN_CVES, WEIGHTS).

2.2 Tipado Estático y Modelado de Datos

- Anotaciones de Tipo (Type Hints): Se deben definir los tipos de datos en la firma de cada función o método público para facilitar la depuración estática y autocompletado.

```
`python
def calculate_score(findings: Iterable[Finding]) -> int:
`
```

- Modelos de Datos:
- Usar dataclasses (en app/models.py) para representar las entidades del dominio de forma ligera.
- Usar Pydantic (en app/schemas.py) para la validación y serialización de los payloads de entrada/salida en la API REST.

2.3 Arquitectura y Organización del Código

- Desacoplamiento de Capas: Las rutas (app/routers/) sólo deben encargarse de la validación inicial del protocolo HTTP/API y la delegación. La lógica de auditoría pesada debe delegarse a los servicios independientes en app/services/.
- Manejo del Estado y Concurrencia: La persistencia y el estado de la aplicación se gestionan de forma centralizada a través de app/store.py. Al ser un servidor web asíncrono multiproceso, todas las operaciones de escritura y lectura en variables de memoria del servidor deben protegerse con un bloqueo de exclusión mutua (threading.Lock) para evitar condiciones de carrera (Race Conditions) en SQLite.

3. Estándares para la Herramienta de Línea de Comandos (CLI)

Dado que el sistema incluye una CLI en la raíz (cli.py), se deben seguir las siguientes directrices:

- Librería de Parámetros: Usar la biblioteca estándar argparse para definir los parámetros y banderas. Las banderas comunes deben ser descriptivas (ej: --json para exportar en formato JSON, --url para pasar un endpoint, etc.).
- Salida de Terminal Dinámica:
- El formato de texto de salida debe ser amigable para humanos por defecto, con indentaciones y separadores.
- Se debe utilizar soporte para colores ANSI de terminal para alertar sobre la severidad de hallazgos (ej. Rojo para severidad alta, Amarillo para severidad media y Azul para baja).
- La opción --json debe silenciar las impresiones decorativas y devolver un arreglo JSON válido en stdout para facilitar integraciones en tuberías CI/CD.
- Códigos de Salida (Exit Codes):
- Retornar 0 si el análisis se ejecutó correctamente y no se superó el umbral crítico de fallos de seguridad.
- Retornar 1 si existieron fallos de ejecución, errores de parámetros, o fallos de red.
- Retornar 2 si el score de cumplimiento OWASP del código auditado cayó por debajo del límite de seguridad mínimo configurado.

4. Estándares para la Extensión de VS Code (JavaScript)

4.1 Estilo de JavaScript

Dado que la extensión se desarrolla en JavaScript Vanilla para mantenerla ligera y autocontenida:

- Variables y Constantes: Se debe priorizar el uso de `const` para referencias que no cambian de valor y `let` para las variables mutables. El uso de `var` está desaconsejado.
- Nombres de funciones y variables: En formato camelCase (ej: `updateStatusBar()`, `currentDiagnostics`).
- Nombres de constantes: En formato SNAKE_CASE (ej: `PYTHON_PATH_SETTING`).
- Manejo de Promesas: Se debe utilizar la sintaxis `async/await` en lugar de encadenar múltiples `.then()` para asegurar la legibilidad en procesos asíncronos (como el llamado al ejecutable de Python o a la API).

4.2 Arquitectura de la Extensión

- Ciclo de Vida y Eventos: El archivo `extension.js` debe inicializar recursos únicamente en el método `activate(context)`. Se deben registrar las suscripciones de los comandos y diagnostic collections para que se liberen limpiamente en `deactivate()`.
- Comunicación Webview-Host: La comunicación entre el panel del Dashboard (HTML renderizado en Webview) y el Host de VS Code debe canalizarse exclusivamente a través del paso de mensajes seguro (`vscode.postMessage` y `window.addEventListener('message')`).

5. Estándares para el Frontend (HTML y CSS)

5.1 HTML

- HTML5 Semántico: Usar etiquetas descriptivas como `<header>`, `<nav>`, `<main>`, `<section>`, `<article>` y `<footer>` en lugar de agrupar todo en contenedores genéricos `<div>`.

- **Accesibilidad:** Todos los elementos interactivos (botones, inputs, enlaces) deben contar con atributos claros y etiquetas descriptivas. Los campos del formulario deben estar conectados a sus `<label>` correspondientes.

5.2 CSS

- **Sin Librerías / Frameworks:** Todo el diseño estético de la aplicación web y de la extensión debe realizarse con CSS Vanilla, evitando TailwindCSS o Bootstrap, lo que reduce la latencia y elimina dependencias.
- **Variables de CSS:** Definir un sistema de tokens de diseño globales (colores de fondo, bordes, espaciados y tipografía) en el bloque `:root`.
- **Diseño Glassmorphism y Rich Aesthetics:**
- Emplear sombras suaves basadas en HSL (`box-shadow: 0 8px 32px 0 hsla(...)`).
- Utilizar el filtro de desenfoque de fondo (`backdrop-filter: blur(12px)`) para lograr el efecto esmerilado de vidrio.
- Asegurar la responsividad total utilizando Flexbox y CSS Grid.

6. Manejo de Errores y Registro (Logging)

- **Manejo de Excepciones:** No se permiten bloques `except: pass` vacíos. Toda excepción debe capturarse de forma específica (ej. `except sqlite3.Error` en vez de `except Exception`) y registrarse apropiadamente.
- **Formato y Niveles de Logs:**
- Usar la librería estándar logging en Python.
- Los logs deben incluir el timestamp en formato ISO 8601, el nivel de log y el mensaje.
- Usar los niveles correctos: INFO para flujo normal de ejecución, WARNING para fallos recuperables o degradaciones del sistema, ERROR para fallos de solicitudes o base de datos que afectan al usuario, y CRITICAL para problemas que detienen la aplicación.
- **Respuestas de API:** En caso de error, la API de FastAPI debe responder con los códigos de estado HTTP correctos (400 para parámetros inválidos, 401 para credenciales/tokens incorrectos, 404 para escaneos inexistentes, 500 para fallos del servidor) y un payload JSON estructurado con la llave `detail` especificando la razón del fallo.

7. Control de Seguridad en la Programación (OWASP Aplicado)

Dado que este sistema es un verificador de seguridad, debe modelarse con un alto nivel de resiliencia:

- Inyección SQL: Queda estrictamente prohibido concatenar variables directas en sentencias de consulta. Se debe utilizar la parametrización de consultas nativas de `sqlite3` o `pymysql` (ej: `cursor.execute("SELECT * FROM users WHERE username = %s", (username,))`).
- Inyección de Comandos: Al ejecutar procesos secundarios de OS (como correr comandos `Git` o interactuar con `cli.py`), usar listas para argumentos (`subprocess.run(["git", "clone", url])`) en lugar de strings sueltos (`subprocess.run(f"git clone {url}")`), y desactivar la bandera `shell=True` para evitar la manipulación de variables externas.
- Secretos y Tokens: Nunca almacenar tokens de prueba, llaves de API o contraseñas de bases de datos directamente en el código fuente. Se debe utilizar un archivo `.env` local para configurar las variables de entorno y consumirlas vía `os.getenv()`.

8. Pruebas y Aseguramiento de Calidad

8.1 Pruebas Unitarias

- Cada nueva funcionalidad, regla del escáner o ruta del API debe contar con pruebas unitarias implementadas en el directorio `/tests`.
- Se utiliza el framework `pytest`. Para ejecutar la suite de pruebas localmente se debe invocar:

```
`bash
python -m pytest -q
```

8.2 Estándar de Mensajes de Commit (Git)

Se recomienda seguir la especificación de commits convencionales para facilitar el seguimiento del historial de cambios:

- feat: incorporación de una nueva funcionalidad (ej. feat: agregar analisis de repositorio github).
- fix: corrección de un error o vulnerabilidad (ej. fix: corregir inyeccion de comandos en ejecutor python).
- docs: modificaciones o adición de documentación (ej. docs: crear diccionario de datos).
- refactor: cambios en el código que no alteran el comportamiento del sistema.